

Software Engineering

CE2006/CZ2006

Requirements Elicitation 3 of 3

Discussion Topics

- Use Case description
- Review of requirement elicitation process
- Software Requirements Specification (SRS)
- Reading
 - Bruegge Chap 4.4
 - Fowler Chap 9

Use Case Model

Use Case Diagram (static)

+

Use Case Description (dynamic)

- Provides coherent visual and textual description of system functionalities
- In use-case-driven iterative development, prioritise use cases for implementation

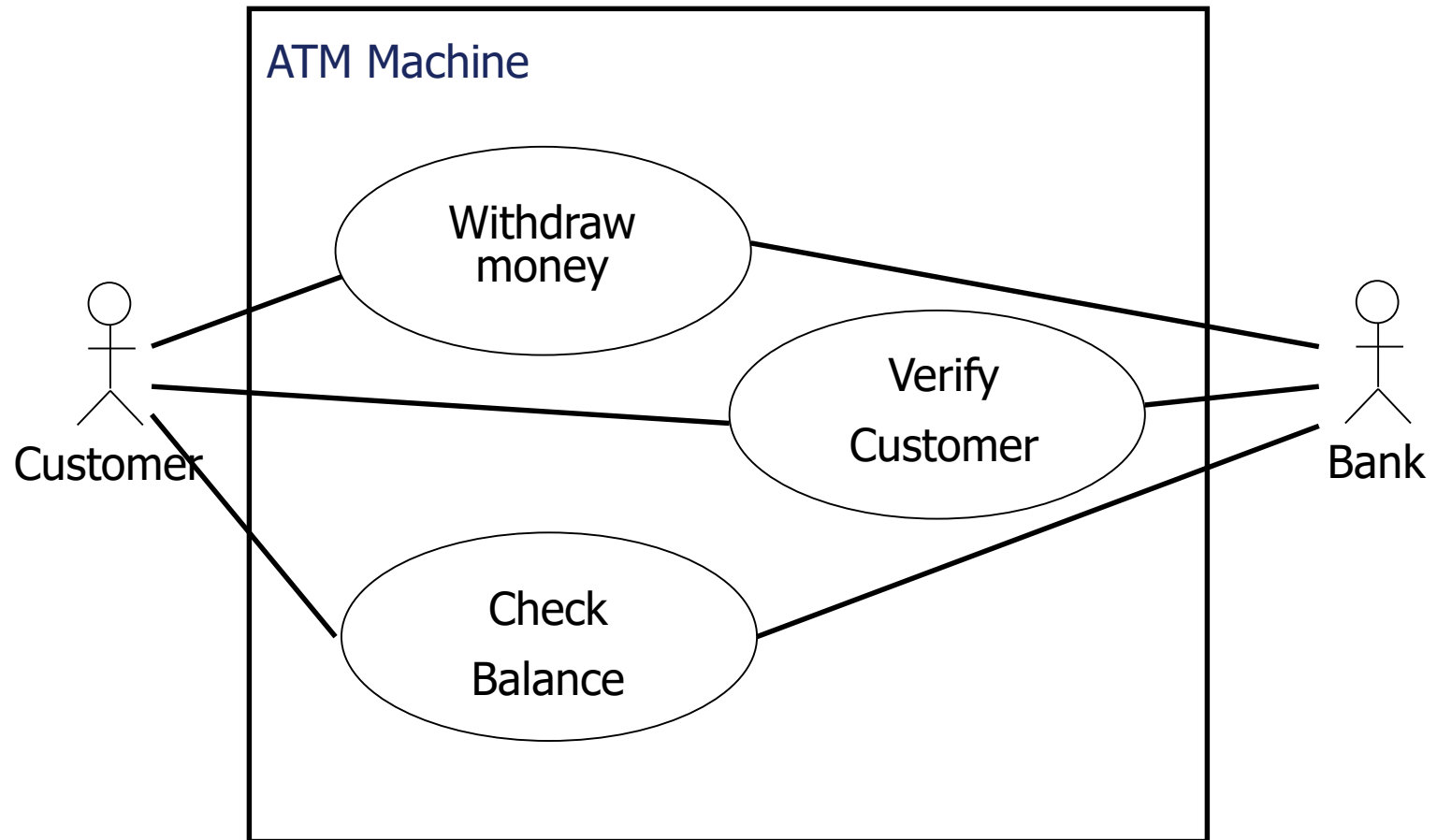
Each Use Case Must have a Use Case Description that contains

- Participating Actors (Initiating Actor)
 - A use case can have more than one participating actors
 - Initiating actor triggers the use case
 - If the use case is triggered by a system event or another use case, the initiating actor is NULL
- Entry conditions (Pre-condition)
 - The system state must be satisfied to execute a use case
- Exit Conditions (Post-condition)
 - The system state must be reached after executing a use case

Each Use Case Must have a Use Case Description that contains

- Flow of Events (a.k.a Normal or Main Successful Flow)
 - Step #1 – How Actor triggers the use case
 - Step #2 – How System responds
 - Step #3 – ...
- Alternative Flows
 - Variations or errors in the interaction (e.g., wrong username/password)
 - Return back to the normal flow of events
- Exceptions
 - Exceptional situations that cause the failure of the use case (e.g., network not available)

Example of Use Case Diagram



Example #1 Actor, Entry/Exist Conditions

The Bank Customer **Withdraw Money** Using ATM

Participating actor:

Bank Customer (Initiating Actor), Bank

Entry Conditions:

1. Bank Customer has a Bank Account with the Bank
- and***
2. Bank Customer has an ATM Card and PIN

Exit Conditions:

1. Bank Customer receives the requested cash
- or***
2. Bank Customer receives an explanation from the ATM why the cash could not be dispensed

Example #1 Flow of Events (Normal Flows)

Actor steps

1.The Bank Customer inputs the card into the ATM

3. The Bank Customer enters a PIN

6. The Bank Customer enters an amount

9. The Bank Customer removes card, receipt and money

System steps

2.The ATM requests the input of a six-digit PIN.

4. The ATM verifies the card and PIN with the account information in the Bank

5.If the card and PIN is verified the ATM requests the amount to withdraw

7. The ATM verifies the amount is available in the customer account

8. If the customer account has sufficient funds, the ATM outputs the money and card and a receipt

Example #1 Flow of Events (Normal Flows)

- 1.The Bank Customer inputs the card into the ATM
- 2.The ATM requests the input of a six-digit PIN.
3. The Bank Customer enters a PIN
4. The ATM verifies the card and PIN with the account information in the Bank
- 5.If the card and PIN is verified the ATM requests the amount to withdraw
6. The Bank Customer enters an amount
7. The ATM verifies the amount is available in the customer account
8. If the customer account has sufficient funds, the ATM outputs the money and card and a receipt
9. The Bank Customer removes card, receipt and money

Example #1 Alternative Flows

AF-S5: If the card and PIN are invalid

1. The ATM displays the message "Invalid card and PIN. Please try again!" for 2 seconds
2. The ATM returns to the step 2

AF-S8: If the customer account has insufficient funds

1. The ATM displays the message "Insufficient funds in your account! Please enter a smaller amount!" for 2 seconds
2. The ATM returns to the step 6

Example #1 Exceptions

EX1: If the Bank Customer enters invalid PIN for three times

1. The ATM displays the message "Card is suspended! Please call the customer service (1633) to reactive the card" for 2 seconds
2. The ATM outputs the card

EX2: If the ATM has insufficient funds

1. The ATM displays the message "This ATM is off-service due to insufficient funds!" at the start screen
2. The ATM does not accept the card

Example #2 Course Registration System

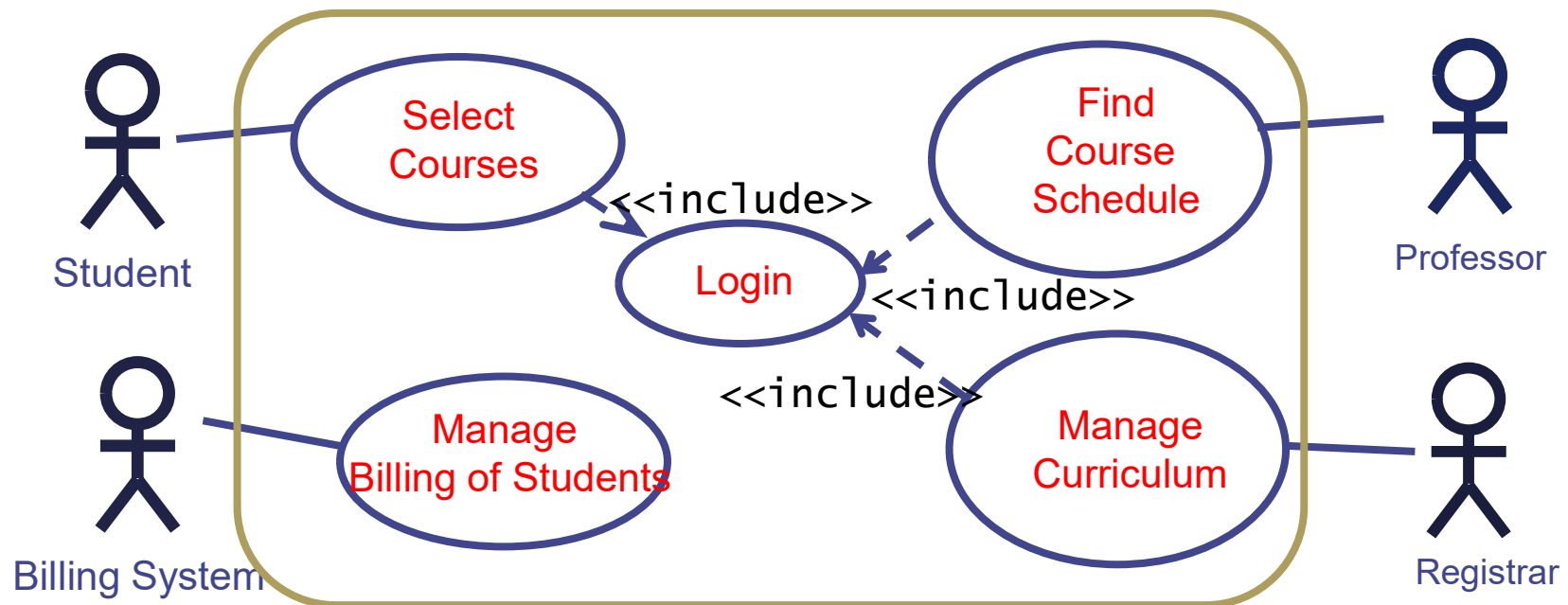
- Registrar manages the course curriculum for a semester
- Students select 4 primary courses and 2 alternate courses
- Students use the system to add and drop courses till the end of the second week of the semester
- Once a Student registers courses for a semester, the registration system notifies the Billing System so the student may be billed for the semester
- Professors use the system to find their course schedule
- All users of the registration system must login the course registration system with the assigned password **before** performing the tasks

Example #2 Course Registration System

- Registrar manages the course curriculum for a semester
- Students select 4 primary courses and 2 alternate courses
- Students use the system to add and drop courses till the end of the second week of the semester
- Once a Student registers courses for a semester, the registration system notifies the Billing System so the student may be billed for the semester
- Professors use the system to find their course schedule
- All users of the registration system must login the course registration system with the assigned password **before** performing the tasks

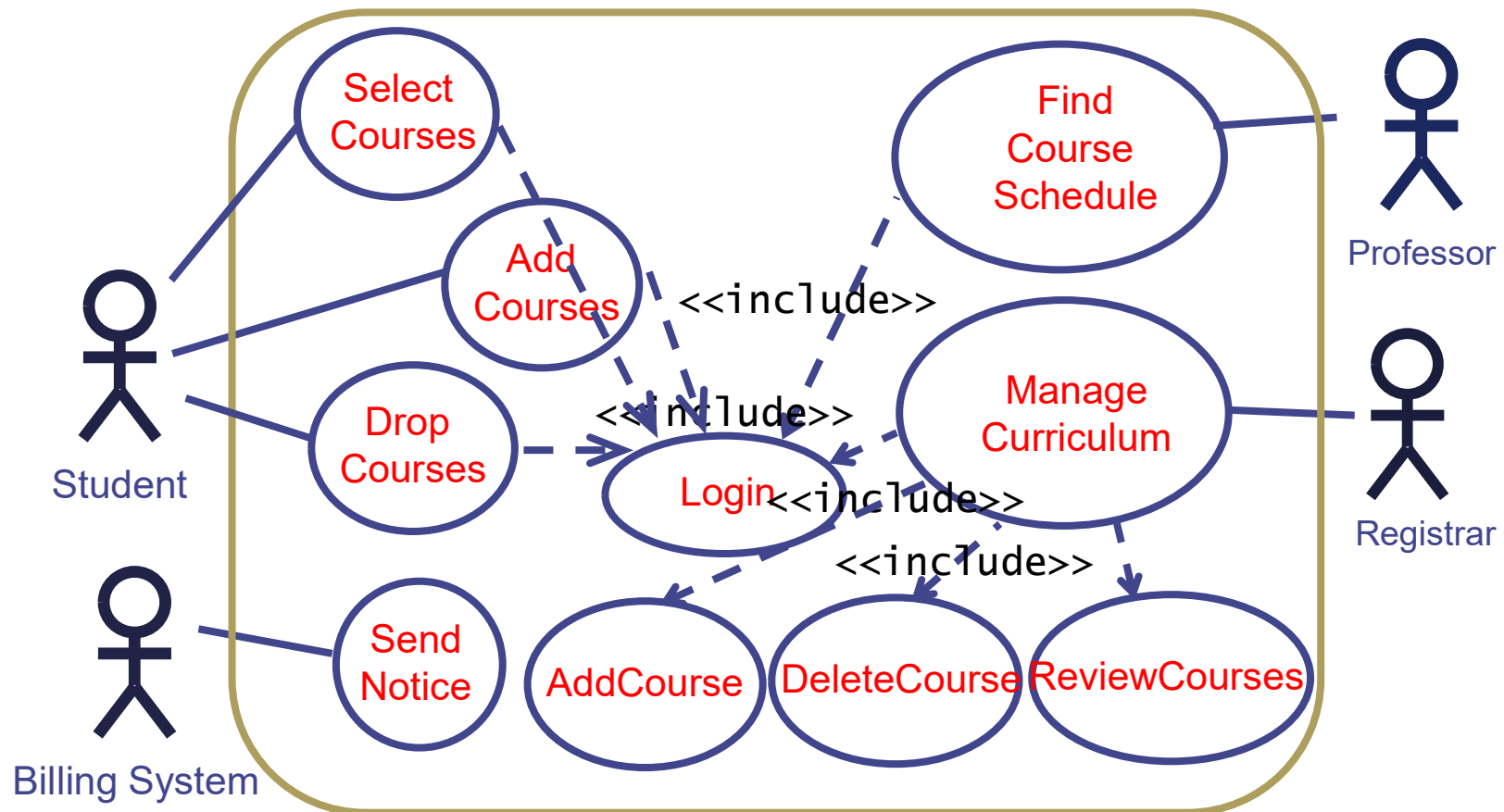
Example#2 Use Case Diagram

- Actors are identified and Use Case diagram are created to visualize the relationships between actors and Use Cases



Example#2 Use Case Diagram

- More use cases



Example#2 Use Case Description:

Manage Curriculum - Flow of Events

1. The system uses the included use case Login to verify the Registrar.
2. On successful login, the system prompts the Registrar to select the current semester or a future semester.
3. The Registrar selects the desired semester.
4. The system prompts the Registrar to select the desired activity: ADD, DELETE, REVIEW, or QUIT.
5. If the Registrar selects the activity ADD, then he uses the included use case AddCourse to add a course.
6. If the Registrar selects the activity DELETE, then he uses the included use case DeleteCourse to delete a course.
7. If the Registrar selects the activity REVIEW, then he uses the included use case ReviewCourses to review existing courses.
8. If the Registrar selects the activity QUIT, the system returns to the semester selection screen.

A Use Case Template should be used to describe each Use Case

See the Use Case Description Template available on NTULearn course website.

Use Case ID:			
Use Case Name:			
Created By:		Last Updated By:	
Date Created:		Date Last Updated:	

Actor:	
Description:	
Preconditions:	
Postconditions:	
Priority:	
Frequency of Use:	
Flow of Events:	
Alternative Flows:	
Exceptions:	
Includes:	
Special Requirements:	
Assumptions:	
Notes and Issues:	

Review of Requirement Elicitation Process

Key Concepts and Deliverables

- Functional requirements
- Use case model
 - Use case diagram
 - Use case description
- Non-functional requirements
- Data dictionary (problem domain glossary)
- UI prototype

Steps 1. Project Team

Review clients specification and review any legacy or competitive products (if available) to:

1. Atomise requirements.
2. Identify Actors
3. Identify main Use Cases (Functional Requirements)
4. Identify Non-Functional requirements
5. Initiate generation of Data Dictionary
6. Identify uncertainties to clarify with client (TBD – To Be Determined). Do not guess the user requirements.

Step 2. Project Team meet Client

(Choose appropriate meeting - Interview, Observation, Workshop)

The agenda and necessary outcomes of the meeting must be set and agreed before the meeting.

Do not waste the Project Teams or Clients time in unstructured unproductive meetings.

Typical Agenda

1. Present and walk through the initial requirements and Data Dictionary. Get agreement.
2. Clarify uncertainties (the TBD issues).
3. Discuss any new requirements the client identifies.

Repeat Steps 1 & 2.

Project Team and Client

The refining process continues iteratively until all uncertainties and ambiguities of the functional and non-functional requirements and the definition of terms in the data dictionary are agreed with the clients.

A prototype demonstrating the appearance or user interface and dialogues of the functionality may be produced to show the intended functionality.

Try to minimize number of meetings by making meetings productive.